



Sincronização de Servidores Cloud com o CMDB

Relatório executivo • AWS, Azure e GCP

AUTOMAÇÃO	DESTINO	ESCOPO
Ansible Automation Platform (AAP)	Jira Assets (CMDB)	Ciclo de vida das VMs

Objetivo

Manter o cadastro corporativo de servidores alinhado ao estado real das VMs nas Clouds, com regras de segurança que limitam o que a automação pode criar, atualizar, reativar ou desativar.

● CRIAR ● ATUALIZAR ● SKIP ● REATIVAR ● DESATIVAR ● PROTEGER

Documento preparado para apresentação, operação e manutenção do código.

1. Resumo executivo

Os três projetos funcionam como uma ponte entre os ambientes de Cloud, o AAP e o Jira Assets (CMDB). O AAP consulta os inventários, transforma os dados para um formato comum, compara o estado atual com o cadastro corporativo e executa a ação correspondente.

Situação	Cloud	CMDB	Resultado
Novo servidor	Existe	Não existe	● CRIAR
Resize	t3.large	t3.micro	● ATUALIZAR
Sem mudança	Igual	Igual	● SKIP
Servidor voltou	Existe	Desativado	● REATIVAR
Servidor removido	Não existe	Em uso	● DESATIVAR

A automação funciona como uma reconciliação controlada entre a Cloud e o CMDB.

2. A lógica em uma visão simples

Etapa	O que acontece	Por que
1 • Inventário	AAP recebe os hosts da Cloud	Descobrir o estado real
2 • Transformação	Filters Python organizam os dados	Padronizar os valores
3 • Consulta	Busca o CI correspondente no CMDB	Saber se já existe
4 • Comparação	Cloud × CMDB	Identificar diferença
5 • Decisão	CREATE / UPDATE / SKIP / REACTIVATE / DEACTIVATE	Escolher a ação
6 • Execução	AAP altera o CMDB	Sincronizar o cadastro

Em linguagem de negócio: a Cloud informa o que existe; o AAP aplica as regras; o CMDB recebe a informação corporativa.

3. Governança: o Ansible só toca no que ele mesmo criou

Para este processo, CI gerenciado pelo Ansible significa, de forma objetiva, um CI que possui Last User = Ansible. Esse campo é o indicador usado para saber que aquele servidor foi criado pela própria automação.

Last User do CI	VM sumiu da Cloud	VM voltou à Cloud
Ansible	● Pode desativar	● Pode reativar
Outro valor / vazio	● Não alterar	● Não alterar

Assim, a regra é simples: o Ansible só deve tocar nos servidores que ele mesmo criou — aqueles cujo Last User é Ansible. Um CI criado por outra equipe não entra automaticamente no ciclo de desativação ou reativação.

Isso é especialmente importante porque o CMDB pode conter servidores que existem por outros motivos ou são administrados por outros times.

4. Proteção de recursos de cluster

Cloud	Recurso protegido	Objetivo
AWS	EKS	Evitar tratamento de nós de cluster como VMs comuns
Azure	AKS	Evitar tratamento de hosts AKS como VMs comuns
GCP	GKE	Evitar desativação indevida de nós GKE

As proteções de cluster são independentes de Last User. Uma camada identifica recursos especiais; a outra controla a responsabilidade da automação.

5. Alteração de tamanho da VM

Quando uma VM sofre resize, o recurso continua sendo o mesmo servidor, mas seu modelo técnico muda. Exemplo: uma EC2 que era t3.micro passa a ser t3.large.

Momento	Cloud	CMDB	Resultado
Antes	t3.micro	t3.micro	Sem diferença
Resize	t3.large	t3.micro	Diferença detectada
Depois	t3.large	t3.large	● ATUALIZADO

O conceito é equivalente em Azure e GCP: o tipo/tamanho da VM é coletado, traduzido pelo mapa do projeto e comparado com o Modelo do Servidor registrado no CMDB.

6. Manutenção de modelos de servidor

Cloud	Campo da Cloud	Arquivo de mapa	Exemplo
AWS	instance_type	modelo_servidor_map_aws.yml	t3.large → GDA-...
Azure	vm_size	modelo_servidor_map_azure.yml	Standard_D4s_v3 → GDA-...
GCP	machine_type	modelo_servidor_map_gcp.yml	e2-medium → GDA-...

Quando surgir um modelo novo:

Passo	Procedimento
1	Identificar exatamente o valor recebido da Cloud.

Passo	Procedimento
2	Verificar se o modelo já existe no CMDB.
3	Se não existir, pedir o cadastro ao time de CMDB.
4	Receber o objectKey / GDA oficial.
5	Adicionar o par modelo → GDA ao mapa correto.
6	Revisar CPU/RAM no mapa de especificações, quando aplicável.
7	Testar com poucos servidores e dry run.

Regra importante: não inventar um GDA. Se o valor não existir no CMDB, o ID oficial deve vir do responsável pelo cadastro.

7. Como adicionar um novo campo

Componente	Responsabilidade	Exemplo
mapeamento_*_cmdb.yml	Relaciona o campo ao atributo do CMDB	Criticidade → criticidade_cloud
Filter Python	Produz o valor	criticidade_cloud = Alta
main.yml	Orquestra a comparação	Decide UPDATE/SKIP
Tasks específicas	Executam ações	Criar / reativar / desativar

Regra prática: o mapping responde “onde colocar?”; o filter responde “qual valor enviar?”.

8. Principais problemas e resposta

Problema	Causa provável	Resposta
Modelo não existe	Não há objeto correspondente no CMDB	Pedir cadastro ao time CMDB e receber GDA
Modelo existe, mas não atualiza	Mapa/atributo/comparação	Validar GDA e atributo Modelo do Servidor
CPU/RAM não aparecem	Tipo ausente no mapa de specs	Cadastrar especificação
Campo novo não aparece	Mapping/filter incompleto	Verificar chave e geração do valor
Cluster aparece para ação	Proteção/inventário	Não executar em produção até validar
Desativação inesperada	CI não deveria estar sob gestão	Conferir Last User e elegibilidade
Muitas alterações	Execução ampla sem validação	Usar dry run

9. Estratégia de testes

O teste full com `dry_run` é o principal mecanismo de validação. Ele permite enxergar previamente quais servidores seriam criados, atualizados, reativados ou desativados, sem efetivar as alterações.

Objetivo	Prática recomendada
Novo modelo	Testar uma VM que utilize exatamente o novo tipo
Visualizar mudanças	Executar o fluxo full com <code>dry_run</code>
Reativação	Conferir a lista de elegíveis no <code>dry_run</code>
Desativação	Conferir a lista de elegíveis antes da ação
Produção	Executar somente após validar o resultado do <code>dry_run</code>

Primeiro descobrir → depois validar → só então alterar. O `dry_run` do fluxo full deve ser o caminho normal de conferência.

10. Guia rápido para quem mantém o código

Necessidade	Onde olhar primeiro
Novo modelo AWS	<code>modelo_servidor_map_aws.yml</code> + specs
Novo modelo Azure	<code>modelo_servidor_map_azure.yml</code> + <code>azure_vm_specs</code>
Novo modelo GCP	<code>modelo_servidor_map_gcp.yml</code> + <code>gcp_machine_specs</code>
Novo atributo CMDB	<code>mapeamento_*_cmdb.yml</code> + filter Python
Alterar decisão	<code>tasks/main.yml</code>
Alterar criação	<code>tasks/create.yml</code>
Alterar reativação	<code>tasks/reactivate.yml</code>
Alterar desativação	<code>tasks/deactivate.yml</code>

11. Perguntas esperadas

Pergunta	Resposta
A automação desativa qualquer CI que sumiu?	Não. Ela só deve desativar CIs cujo Last User seja Ansible e que também estejam dentro das demais regras de elegibilidade.
O que significa CI do Ansible?	É o CI cujo campo Last User está preenchido com Ansible — ou seja, um servidor criado pela própria automação.
EKS/AKS/GKE podem ser desativados?	Existem proteções específicas para impedir tratamento de nós de cluster como VMs comuns.
O que acontece quando a VM muda de tamanho?	A diferença pode ser detectada e o Modelo do Servidor do CI gerenciado pode ser atualizado.
Quem cria um novo GDA?	O time responsável pelo CMDB. A automação usa o ID oficial.

Pergunta	Resposta
Como testar sem risco?	Executar o fluxo full com dry_run e revisar as listas antes da execução efetiva.

12. Conclusão e checklist

A solução transforma os inventários de AWS, Azure e GCP em uma rotina controlada de sincronização com o CMDB. O desenho separa coleta, transformação, decisão e execução, permitindo manutenção e evolução sem reescrever todo o processo.

O princípio central de governança é: o Ansible só deve alterar servidores que ele mesmo criou, identificados no CMDB por Last User = Ansible. As proteções de cluster adicionam uma camada complementar de segurança.

	Verificação
●	Novo modelo identificado corretamente
●	Modelo existe no CMDB ou foi solicitado ao responsável
●	GDA/objectKey oficial recebido
●	Mapa atualizado
●	CPU/RAM revisados
●	Fluxo full executado em dry_run
●	Lista de ações conferida antes da produção
●	Execução efetiva liberada somente após validação